Tugas Gabungan LKP 2-5 - Blockchain Sederhana

Program CLI blockchain sederhana yang mengintegrasikan Modul 2-5 PBO: struktur dasar Java + Scanner/printf/array/control-flow (P2), class/object/constructor (P3), encapsulation + this + static + overloading (P4), inheritance + super + overriding (P5).

Repository (publik): https://github.com/oxelmour/blockchain-sederhana

Pesan komit: "Tugas Gabungan LKP 2-5"

Struktur: Block (superclass) -> BlockTransaction + BlockData (subclass) + Main (CLI)

Isi PDF ini

1. Hasil running lengkap (tambah 4 block + tampil + cari teks + cari angka + keluar).
2. Potongan kode Array Superclass.
3. Potongan kode Method Overloading.
4. Potongan kode Dynamic Binding (override + pemanggilan polimorfik).

1. Hasil Running (lengkap)

Skenario: tambah Transaksi (Bayar kopi) + Data (Sertifikat tanah) + Transaksi (Kirim ETH) + Data (Foto ijazah), tampilkan 4 block, cari teks 'kopi', cari index 1, keluar. Rantai prev-hash tersambung dan total static = 4.

Output java Main (running-output.txt)

```
=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu: Pilih tipe:
1. Transaksi
2. Data
Tipe: Data: Sender: Receiver: Amount: Block transaksi ditambah. Hash: -1610818418 | Total: 1

=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu: Pilih tipe:
1. Transaksi
2. Data
Tipe: Data: Category: Owner: Block data ditambah. Hash: -1043213970 | Total: 2

=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu: Pilih tipe:
1. Transaksi
2. Data
Tipe: Data: Sender: Receiver: Amount: Block transaksi ditambah. Hash: -1763573974 | Total: 3

=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu: Pilih tipe:
1. Transaksi
2. Data
Tipe: Data: Category: Owner: Block data ditambah. Hash: 13752587 | Total: 4

=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu:
=== Daftar 4 Block (Total dibuat: 4) ===
--- Block ke-1 ---
Index: 0
Data: Bayar kopi
Previous Hash: 0
Hash: -1610818418
```

```
sender='Andi', receiver='Budi', amount=50000.0
--- Block ke-2 ---
Index: 1
Data: Sertifikat tanah
Previous Hash: -1610818418
Hash: -1043213970
Category: Dokumen
Owner: Citra
--- Block ke-3 ---
Index: 2
Data: Kirim ETH
Previous Hash: -1043213970
Hash: -1763573974
sender='Doni', receiver='Eka', amount=0.5
--- Block ke-4 ---
Index: 3
Data: Foto ijazah
Previous Hash: -1763573974
Hash: 13752587
Category: Arsip
Owner: Fajar

=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu: Cari berdasar:
1. Index (angka)
2. Data (teks)
Pilih: Masukkan kata kunci data: Ditemukan pada posisi ke-1:
Index: 0
Data: Bayar kopi
Previous Hash: 0
Hash: -1610818418
sender='Andi', receiver='Budi', amount=50000.0

=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu: Cari berdasar:
1. Index (angka)
2. Data (teks)
Pilih: Masukkan index: Ditemukan pada posisi ke-2:
Index: 1
Data: Sertifikat tanah
Previous Hash: -1610818418
Hash: -1043213970
Category: Dokumen
Owner: Citra

=== Menu Blockchain ===
1. Tambah Block
2. Tampilkan Block
3. Cari Block
4. Keluar
Pilih menu: Program selesai.
```

2. Array Superclass - Block[] menampung subclass

Di Main.java, array dideklarasikan bertipe superclass Block, tetapi diisi objek subclass BlockTransaction dan BlockData. Ini syarat penyimpanan objek polimorfik + materi Array (P2).

Main.java - deklarasi array superclass

```
Block[] chain = new Block[10];
int jumlahBlock = 0;
```

Main.java case 1 - isi array dengan subclass (polimorfik)

```
chain[jumlahBlock] = new BlockTransaction(sender, receiver, amount, idx, data, prev);
jumlahBlock++;
chain[jumlahBlock] = new BlockData(idx, data2, prev, category, owner);
jumlahBlock++;
```

Main.java case 2 - akses tiap elemen array

```
for (int i = 0; i < jumlahBlock; i++) {
    System.out.printf("--- Block ke-%d ---%n", (i + 1));
    chain[i].getInfo();
}
```

3. Method Overloading - cariBlock

Dua method bernama sama cariBlock di Main.java, dibedakan oleh tipe parameter: int (cari berdasar index) vs String (cari berdasar kata kunci data). Dipanggil dari menu Cari case 3.

Main.java - overload 1 (int)

```
public static void cariBlock(int index, Block[] chain, int jumlahBlock) {
    boolean ketemu = false;
    for (int i = 0; i < jumlahBlock; i++) {
        if (chain[i].getIndex() == index) {
            System.out.printf("Ditemukan pada posisi ke-%d:%n", (i + 1));
            chain[i].getInfo();
            ketemu = true;
        }
    }
    if (!ketemu) {
        System.out.printf("Block dengan index %d tidak ditemukan.%n", index);
    }
}
```

Main.java - overload 2 (String)

```
public static void cariBlock(String keyword, Block[] chain, int jumlahBlock) {
    boolean ketemu = false;
    for (int i = 0; i < jumlahBlock; i++) {
        if (chain[i].getData().toLowerCase().contains(keyword.toLowerCase())) {
            System.out.printf("Ditemukan pada posisi ke-%d:%n", (i + 1));
            chain[i].getInfo();
            ketemu = true;
        }
    }
    if (!ketemu) {
        System.out.printf("Block dengan kata kunci \"%s\" tidak ditemukan.%n", keyword);
    }
}
```

Main.java case 3 - pemilihan overload

```
if (mode == 1) {
    int key = Integer.parseInt(sIdx);
    cariBlock(key, chain, jumlahBlock);      // panggil versi int
} else if (mode == 2) {
    String key = scanner.nextLine();
    cariBlock(key, chain, jumlahBlock);      // panggil versi String
}
```

4. Dynamic Binding - getInfo() override

Method getInfo() didefinisikan di Block, lalu di-override di kedua subclass dengan @Override. Saat dipanggil lewat referensi superclass (chain[i].getInfo() atau di dalam cariBlock), JVM memilih implementasi subclass saat runtime. Buktinya di hasil running: block transaksi menampilkan sender/receiver/amount, block data menampilkan category/owner.

Block.java - method dasar

```
public void getInfo() {
    System.out.println("Index: " + this.index);
    System.out.println("Data: " + this.data);
    System.out.println("Previous Hash: " + this.previousHash);
    System.out.println("Hash: " + this.hash);
}
```

BlockTransaction.java - override

```
@Override
public void getInfo() {
    super.getInfo();
    System.out.println(", sender='" + sender + '\'' +
            ", receiver='" + receiver + '\'' +
            ", amount=" + amount);
}
```

BlockData.java - override

```
@Override
public void getInfo() {
    super.getInfo();
    System.out.println("Category: " + this.category);
    System.out.println("Owner: " + this.owner);
}
```

Main.java - panggilan polimorfik (binding saat runtime)

```
chain[i].getInfo();  // chain bertipe Block, objek bisa BlockTransaction / BlockData
```